SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Operating Systems **COURSE CODE: CSA0404**

COURSE OUTCOME COVERED

CO5: *Analyze the components and functionalities of Linux operating systems and virtualization environments to enhance their operational efficiency*

Assessment Tool Weightage: 35%

Dr. P. Chandravathi

QUESTIONS: 5 ***Total Marks:25***

Annexure A

CONSTRAINT BASED PROBLEM SOLVING

Code of Conduct:

I SWETHA.V(192524180) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

SWETHA.V

Signature of the student:

Annexure A

CONSTRAINT BASED PROBLEM SOLVING

S.No.	Question	Mark
1	A server with a total storage capacity of 50 GB needs to effectively manage space for rapidly growing system logs, user files, and application data. Analyze the scenario and design an optimized Linux file system layout, incorporating suitable partitioning techniques and storage management tools (such as log rotation), to ensure efficient utilization of disk space and maintain system stability. The solution must ensure that log files do not exceed 10 GB and that critical services continue to operate without interruption due to disk space exhaustion.	6
2	A Linux system handling high CPU workloads suffers from frequent context switching delays. Given that hardware upgrades are not possible and real-time processing must be supported, evaluate kernel-level optimization techniques such as scheduler tuning and the use of kernel modules—to improve overall system performance.	5
3	An administrator must process 1 million log entries per day using only shell scripting. Analyze the problem and design an efficient solution using optimized Linux commands that ensures the script completes within 2 minutes. Justify the choice of commands used for achieving high performance	5
4	A Linux system runs multiple background services, where a faulty process is consuming 90% of CPU resources. Without restarting the system and while ensuring critical services remain unaffected, analyze the situation and propose a process management strategy using appropriate Linux tools to identify, monitor, and control the faulty process.	5
5	A physical server must host 5 virtual machines with a total available RAM of 16 GB. Each VM requires a minimum of 2 GB, while the host operating system needs 4 GB. Analyze the scenario and propose an efficient resource allocation strategy using virtualization concepts to ensure all virtual machines operate smoothly under the given constraints.	4

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints

Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Q1. Optimized Linux File System Layout for a 50 GB Server

Problem Understanding

The server has **50 GB total storage** and must handle:

- Rapidly growing system logs
- User files
- Application data
- Critical system files

The main constraints are:

1. Log files must **not exceed 10 GB**.
2. Critical services must continue running.
3. Disk space must be used efficiently.
4. Disk exhaustion must be prevented.

Proposed File System Layout

Partition	Size	Purpose
/	10 GB	Operating system and system files
/var	10 GB	Logs, cache and variable data
/home	10 GB	User files
/opt	8 GB	Application data
/tmp	2 GB	Temporary files
Swap	4 GB	Virtual memory
Free/Reserved	6 GB	Emergency growth and maintenance
Total	**50 GB**	

Log Management

The /var partition is restricted to **10 GB**, preventing logs from consuming the entire disk.

Linux **logrotate** can be configured to rotate and compress logs regularly.

Example:

sudo nano /etc/logrotate.conf

A sample configuration:

```
/var/log/*.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
}
```

This keeps recent logs while compressing and removing older logs.

Useful commands:

df -h

du -sh /var/log

sudo logrotate -f /etc/logrotate.conf

Storage Monitoring

Disk usage can be monitored using:

df -h

du -sh /var/*

A threshold-based monitoring script can generate an alert when usage reaches, for example, **80–90%**.

Constraint Satisfaction

- /var is limited to **10 GB**, so log storage is controlled.
- Log rotation and compression prevent unnecessary accumulation.
- **6 GB is reserved** for emergency growth.

- Separate partitions prevent logs or user files from consuming the operating system's space.
- Monitoring provides early warning before disk exhaustion.

Q2. Kernel-Level Optimization for High CPU Workloads

Problem Understanding

The Linux system experiences **frequent context switching delays** during high CPU workloads. Hardware upgrades are not possible, and the system must support real-time processing.

The main objectives are:

- Reduce unnecessary context switching.
- Give important processes higher CPU priority.
- Improve real-time responsiveness.
- Avoid affecting critical services.

Proposed Solution

1. Scheduler Tuning

Linux scheduler parameters can be examined using:

sysctl -a | grep sched

CPU scheduling behavior can be tuned according to workload requirements.

Processes can be given appropriate priorities using:

nice -n -5 ./program

For an existing process:

renice -5 -p PID

A lower nice value gives the process higher CPU priority.

2. CPU Affinity

A process can be assigned to specific CPU cores:

taskset -c 0,1 ./program

This reduces unnecessary migration of processes between CPUs.

3. Real-Time Scheduling

For real-time workloads, Linux provides real-time scheduling policies such as:

chrt -f 50 ./program

This gives the process a real-time FIFO scheduling priority.

4. Kernel Modules

Only necessary kernel modules should be loaded.

Check loaded modules:

lsmod

Remove an unnecessary module:

sudo modprobe -r module_name

Reducing unnecessary kernel functionality can lower overhead and improve system efficiency.

Constraint Satisfaction

Constraint	**Solution**
No hardware upgrade	Scheduler and kernel tuning
High CPU workload	CPU affinity and priority tuning
Frequent context switching	CPU affinity and optimized scheduling
Real-time processing	chrt real-time scheduling
System stability	Load only required kernel modules

Q3. Processing 1 Million Log Entries Within 2 Minutes — 5 Marks

Problem Understanding

The administrator must process **1 million log entries per day** using only shell scripting.

Constraints:

- Only shell commands can be used.
- Processing must finish within **2 minutes**.
- Commands must be efficient.

- Repeated unnecessary processing should be avoided.

Efficient Solution

Instead of using slow loops such as:

```
while read line
do
    ...
done
```

use optimized Linux utilities that process large files efficiently.

Example:

```
#!/bin/bash

LOG="/var/log/application.log"

awk '{print $1}' "$LOG" | sort | uniq -c | sort -nr > report.txt
```

This can be used to count occurrences of values such as IP addresses, users, or event types.

For searching specific errors:

```
grep -F "ERROR" "$LOG" > errors.log
```

For counting errors:

```
grep -Fc "ERROR" "$LOG"
```

For extracting fields:

```
awk -F' ' '{print $1, $2, $5}' "$LOG"
```

For compressed logs:

```
zgrep -F "ERROR" /var/log/application.log.gz
```

Recommended Processing Pipeline

```
grep -F "ERROR" application.log | awk '{print $1}' | sort | uniq -c
```

Why These Commands?

Command	Purpose
grep	Fast pattern searching
awk	Efficient field extraction and processing
sort	Organizes data for grouping
uniq -c	Counts repeated entries
zgrep	Searches compressed logs

Performance Strategy

1. Use compiled Linux utilities instead of shell loops.
2. Use grep -F for fixed-string searches.
3. Process only required fields using awk.
4. Avoid creating unnecessary temporary files.
5. Use pipelines to pass data directly between commands.
6. Process only the required log period instead of the entire log history.

Constraint Satisfaction

The solution is designed to process **1 million entries within the 2-minute target** by using optimized utilities and pipelines rather than repeatedly invoking shell commands for each line.

Q4. Managing a Faulty Process Consuming 90% CPU

Problem Understanding

A background process is consuming **90% CPU**. The administrator must:

- Identify the faulty process.
- Monitor its behavior.
- Reduce or stop its CPU usage.
- Keep critical services running.
- Avoid restarting the system.

Step 1: Identify the Process

Use:

top

or:

ps aux --sort=-%cpu | head

This displays processes consuming the highest CPU.

Example:

PID %CPU COMMAND

2456 90.0 faulty_app

Here, **2456** is the suspected process.

Step 2: Monitor the Process

top -p 2456

or:

pidstat -p 2456 1

This helps determine whether the process continuously consumes excessive CPU.

Step 3: Reduce CPU Priority

If the process is required but consuming too many resources:

sudo renice 10 -p 2456

This lowers its CPU priority.

CPU usage can also be restricted using:

sudo systemctl set-property service_name.service CPUQuota=20%

if the process is managed as a systemd service.

Step 4: Stop the Faulty Process

If it is safe to terminate:

kill 2456

If it does not respond:

kill -9 2456

SIGKILL should be used only as a last resort.

Step 5: Protect Critical Services

Before stopping the process:

systemctl status service_name

Critical services should be identified and left untouched.

Constraint Satisfaction

- No system restart is required.
- Critical services remain unaffected.
- The faulty process is identified using top/ps.
- Monitoring is performed using pidstat.
- Priority can be reduced using renice.
- The process can be terminated if necessary.

Q5. Virtualization Resource Allocation for 5 VMs

Problem Understanding

The physical server has:

- Total RAM = **16 GB**
- Number of VMs = **5**
- Minimum RAM per VM = **2 GB**
- Host OS requirement = **4 GB**

Constraint Calculation

RAM required by VMs:

5 × 2 GB = 10 GB

RAM required by host:

4 GB

Total required:

10 GB + 4 GB = 14 GB

Available RAM:

16 GB

Remaining:

16 − 14 = 2 GB

Therefore, the requirements can be satisfied.

Proposed Allocation

Component	RAM
Host OS	4 GB
VM1	2 GB
VM2	2 GB
VM3	2 GB
VM4	2 GB
VM5	2 GB
Total	**14 GB**
Remaining	**2 GB**

Efficient Virtualization Strategy

Each VM receives its required minimum **2 GB**.

The remaining **2 GB** should not be permanently assigned to the VMs. It can be retained as a safety margin for:

- Host OS fluctuations
- Virtualization overhead
- Temporary workload increases
- System stability

Memory ballooning or dynamic memory allocation can be used where supported by the virtualization platform.

Constraint Satisfaction

- All 5 VMs receive at least 2 GB.
- Host receives the required 4 GB.
- Total allocation is only 14 GB.
- 2 GB remains available as a safety buffer.
- No VM receives less than its minimum requirement.